

Extragere imagini bazată pe conținut

Student: Maria Alexandra Moldovan

Grupa 30234

anul 2024

1. Introducere

Proiectul are în vedere realizarea unei aplicații care, pe baza unei imagini selectate și a unui director cu imagini selectat în prealabil, va căuta și afișa 3 imagini cu cel mai mare grad de similitudine comparativ cu imaginea de intrare aleasă. De asemenea, aplicația va calcula și histograme aferente imaginii de intrare și celor 3 imagini găsite.

Gestionarea și căutarea eficientă a imaginilor în colecții mari necesită mult timp, și rezultatele nu sunt întotdeauna cele mai bune, dacă abordăm problema manual. Motivarea constă în necesitatea de a oferi o soluție automată și rapidă pentru identificarea imaginilor similare, economisind timp și efort utilizatorilor. Această aplicație vizează accesul rapid la imagini relevante, și ar putea avea numeroase beneficii practice, precum:

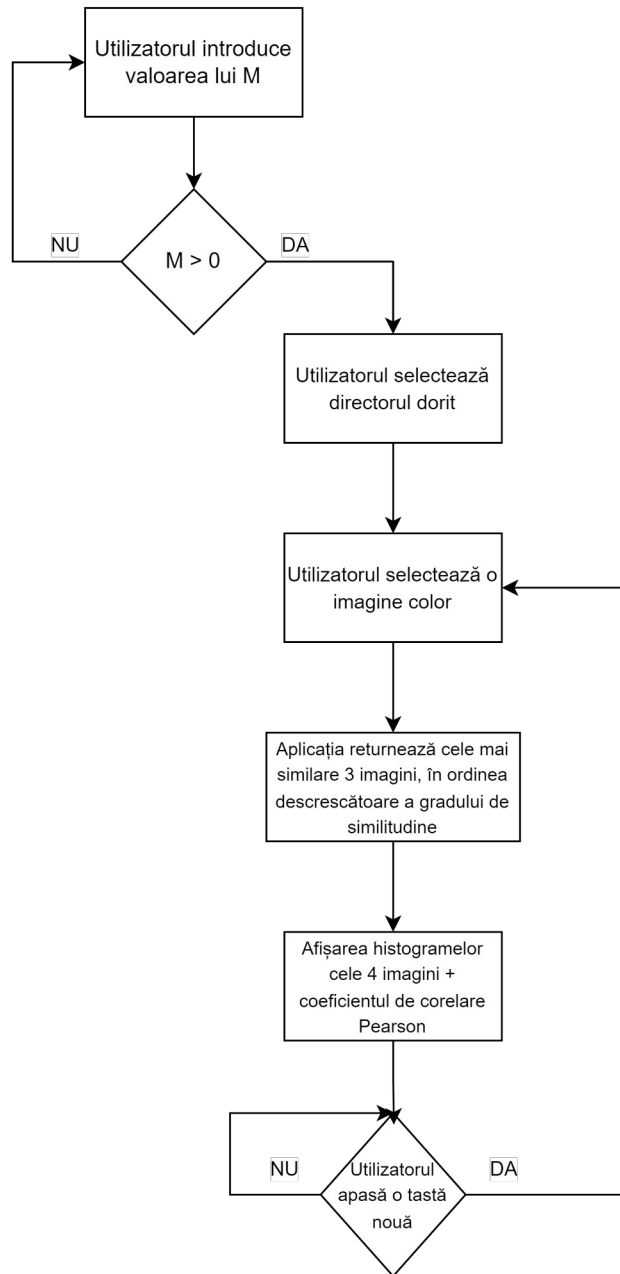
- sortarea imaginilor rezultat de pe motoarele de căutare (ex. Google image search), sau chiar din galeriile foto ale telefoanelor mobile de astăzi, de ex. sortează imaginile în care detectează câini într-un alt director față de cele cu pisici
- cercetare, pentru a găsi rapid imagini similare, de ex. în domeniul medical, pentru a identifica imagini medicale similare pentru a oferi un diagnostic
- security checks: scanare de amprentă/retină pentru acces

2. Considerații teoretice

Termenul de „content-based image retrieval” (CBIR) a fost folosit pentru prima oară în 1992 de către inginerul japonez Toshikazu Kato. De atunci, termenul este utilizat pentru a descrie procesul de extragere al imaginilor dorite dintr-o colecție pe baza unor trăsături specificate.

O tehnică ce va fi folosită este metoda de determinare a similitudinii între imagini color. Aceasta constă în a realiza o conversie RGB $\rightarrow$ HSV pe fiecare zonă a grid-ului/imaginii, iar în urma unor procesări, rezultă un vector V, numit vector de reprezentare al imaginii. Dacă dorim să vedem similitudinea acestei imagini cu o nouă imagine se va calcula pe noua imagine vectorul V corespunzător. Pentru gradele de diferențiere, respectiv de similitudine dintre cele 2 imagini, avem formule diferite, care se bazează pe vectorii corespunzători imaginilor. Ulterior, după calcularea unor histograme, vom determina coeficientul de corelare Pearson (termen derivat din domeniul statisticii), care este cu atât mai ridicat cu cât imaginile sunt mai similare.

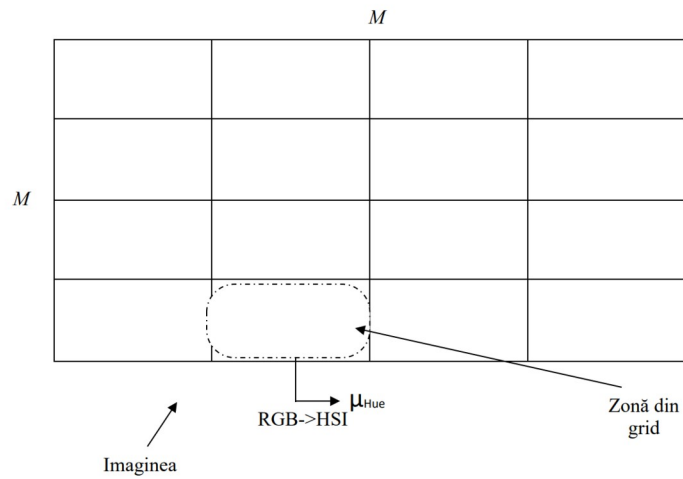
3. Specificații de implementare



Ca si ghid de utilizare, poate fi utilizata ca referinta figura din stanga.

Pentru determinarea celor mai similare 3 imagini, am implementat o metoda prescrisa de similitudine.

Aceasta consta in impartirea imaginii alese intr-un grid de dimensiune M x M (M fiind variabila introdusa de la tastatura), procesarile urmand sa fie facute pe fiecare zona a acestui grid.



Pe fiecare zona a grid-ului, se va realiza o conversie din RGB in HSI. Spatiul de culoare HSI (Hue Saturation Intensity) difera de RGB prin faptul ca nu ne intereseaza procentajul de rosu, verde sau albastru, ci ajustam nuanta, saturatia sau intensitatea.

In urma conversiei, avem nevoie de valoarea H medie pe zona, ignorand zonele cu saturatie egala cu 0, iar aceste valori H vor fi retinute intr-un vector V de dimensiunea grid-ului.

Gradul de diferentiere dintre doua imagini se calculeaza folosind formula:

$$D = \frac{1}{C} \cdot \sqrt{\sum_{i=1}^{M \cdot M} (\min(|V_1[i] - V_2[i]|, 2\pi - |V_1[i] - V_2[i]|)^2, \text{iff } V_1[i] \geq 0 \text{ si } V_2[i] \geq 0)},$$

unde:

V_1, V_2 – vectorii corespunzători celor 2 imagini

$C = \sqrt{N}\pi$ – constantă de normalizare încât D să aibă valori în domeniul $[0,1]$

N = numărul de perechi i pentru care se îndeplinește condiția $V_1[i] \geq 0$ și $V_2[i] \geq 0$

Daca dorim gradul de similitudine dintre doua imagini, se calculeaza folosind formula:

$$S = I - D \text{ daca } N > 0, \text{ altfel } S = 0.$$

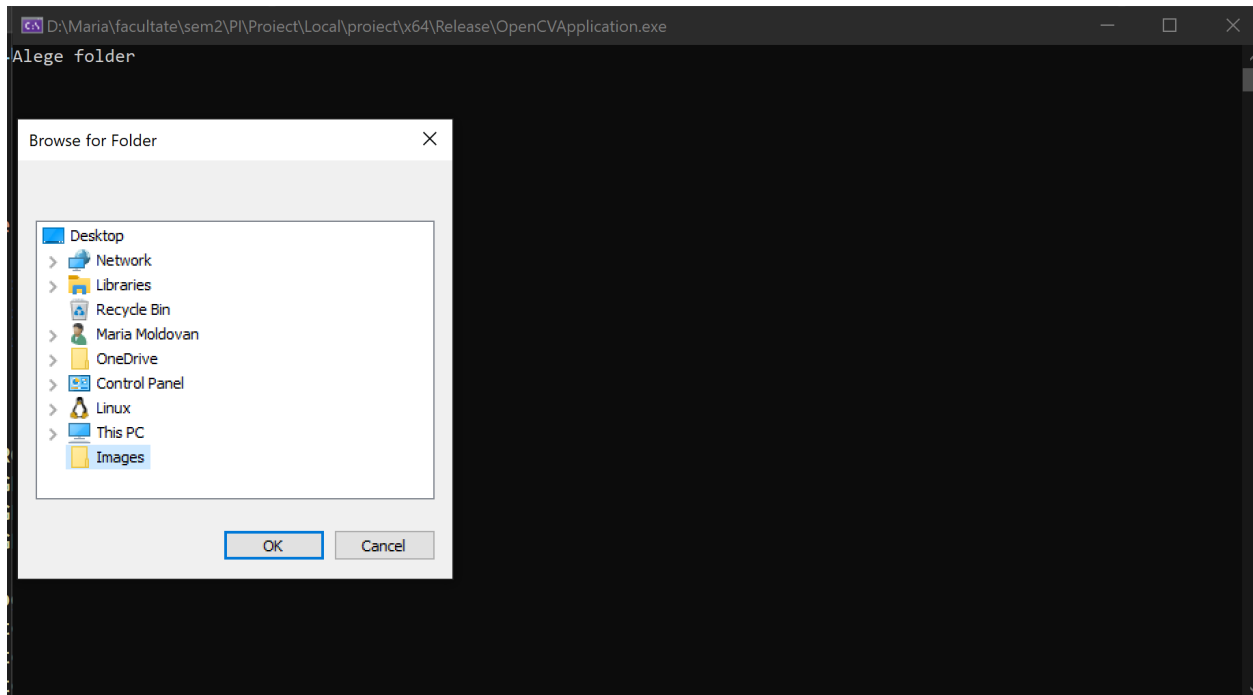
Pentru calcularea coeficientului de corelare Pearson, am folosit urmatoarea formula:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

where

- n is sample size
- x_i, y_i are the individual sample points indexed with i
- $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ (the sample mean); and analogously for $\bar{y}$.

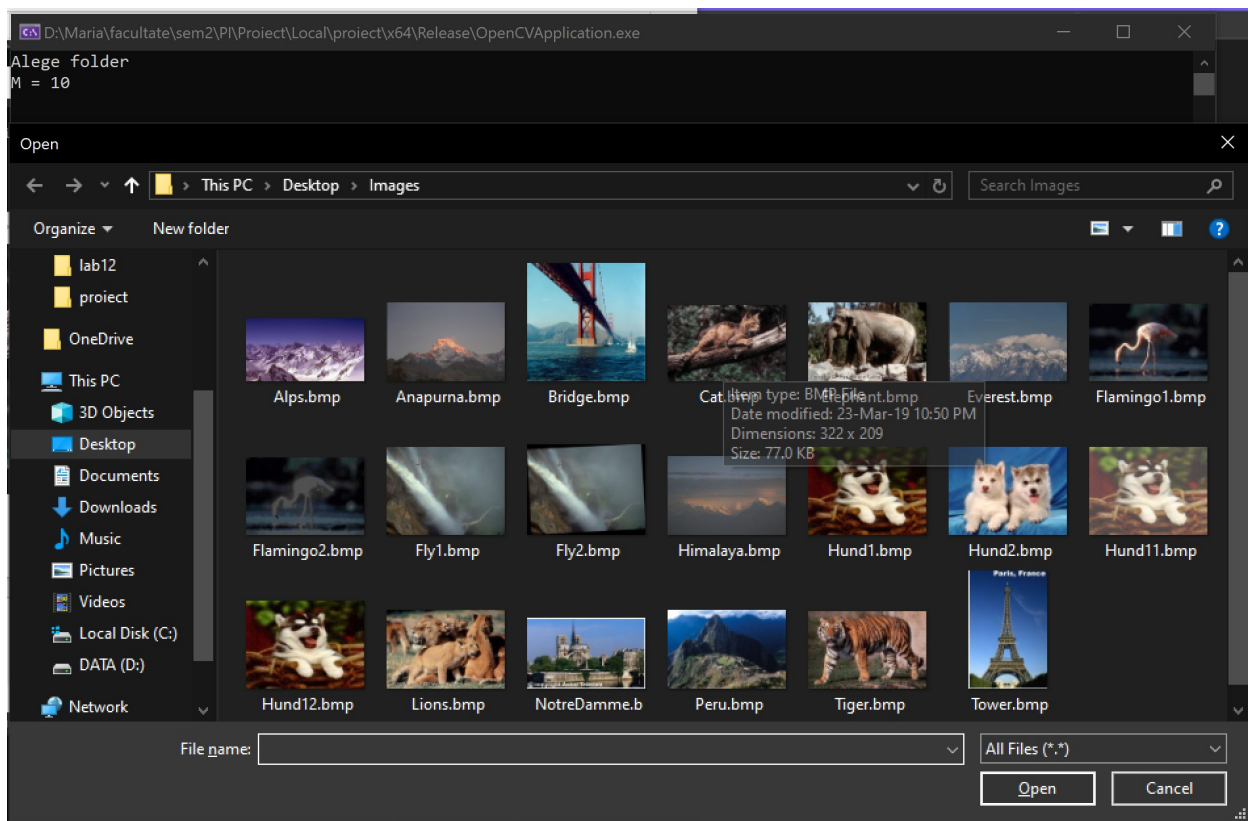
Ca si ghid de utilizare, la rulare utilizatorul alege un folder:



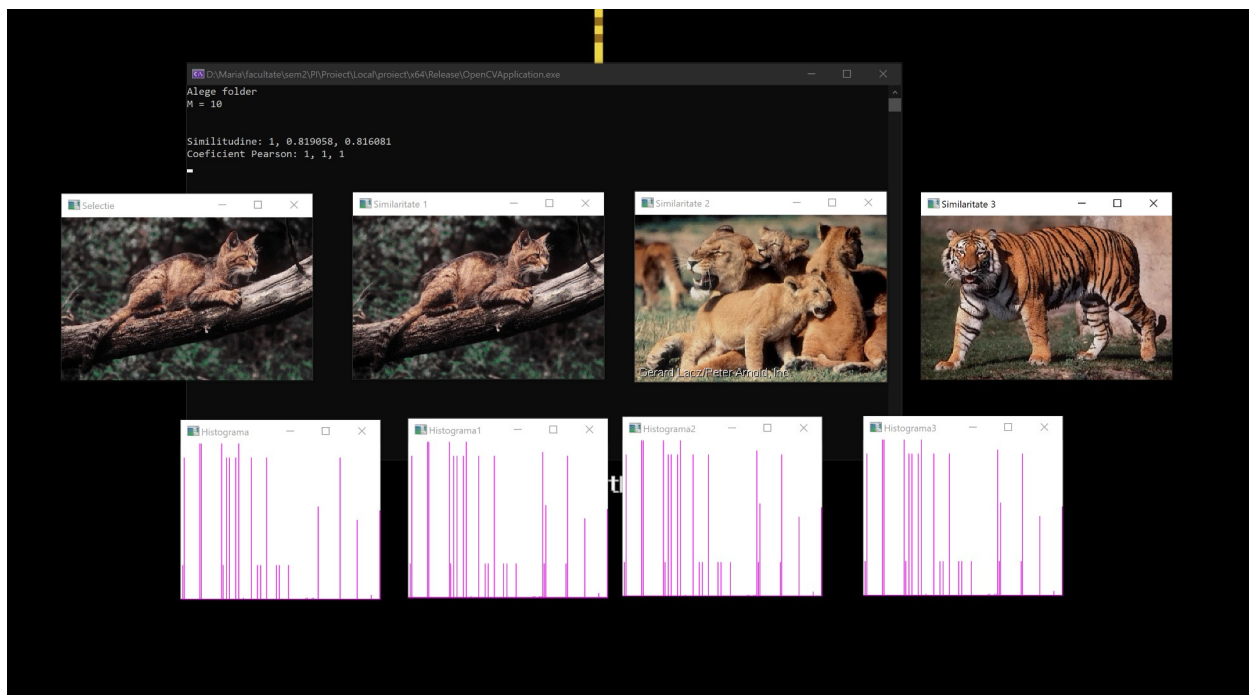
Dupa ce se introduce valoarea M, este validata in cod:

```
int M;
do {
    cout << "M = ";
    cin >> M;
    if (M <= 0) {
        cout << "M trebuie sa fie o valoare strict pozitiva!\n";
    }
} while (M <= 0);
```

Daca M este o valoare valida, se trece la alegerea unei imagini:

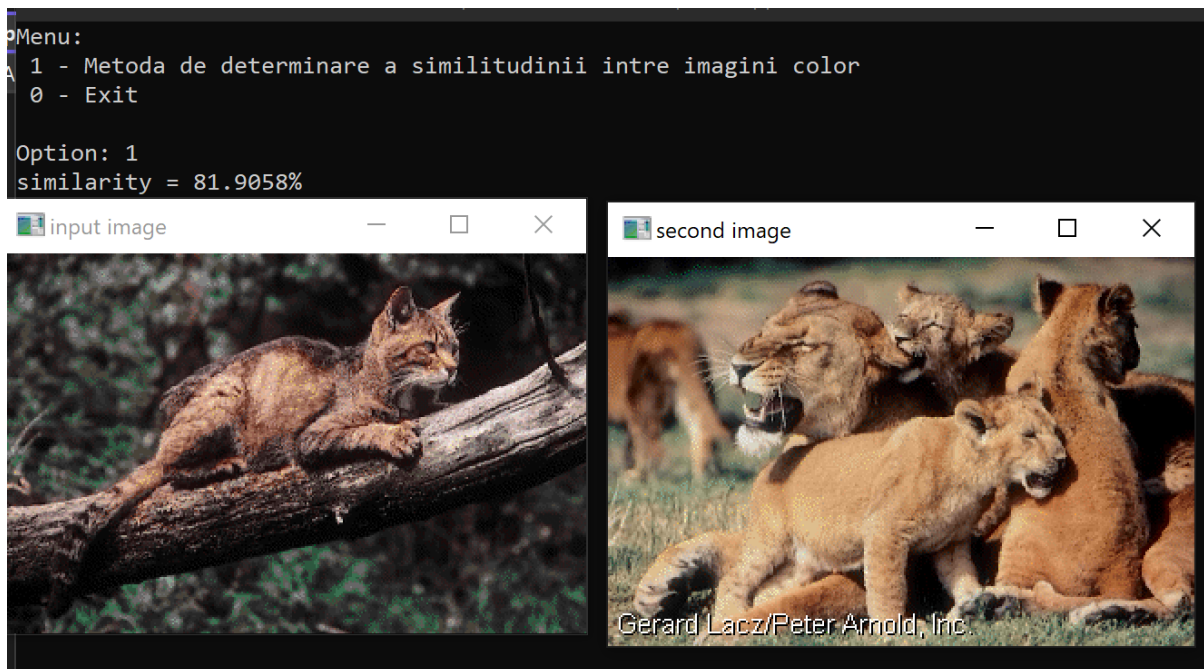
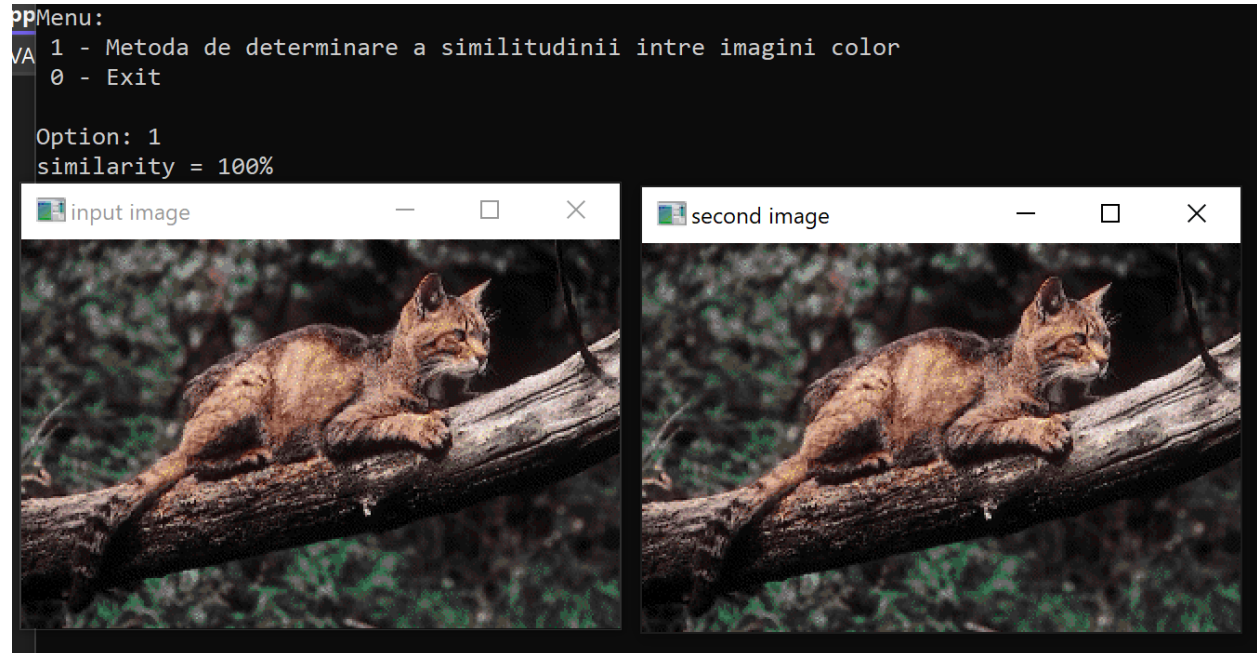


Acesta este rezultatul final. Din pacate, am avut probleme la calcularea histogramelor, asta influentand si rezultatul coeficientului de corelare Pearson, dar cel de similitudine este corect.



4. Rezultate experimentale

- Pentru setul de date, ne vom folosi de imaginile din folder "Images" al folderului proiectului. Utilizatorul va selecta o imagine din acesta, ca mai apoi aplicatia sa afiseze restul.
- În prima faza a proiectului, am testat doar 2 imagini. Pentru implementarea finala, restul algoritmilor nu au funcționat.



5. Concluzii

În timpul realizării proiectului, mi-am aprofundat abilitățile de lucru cu imaginile în OpenCV.

Rezultatele nu au fost pe măsura așteptărilor, având probleme cu crearea histogramelor pe canalul Hue.

Ca și dezvoltări ulterioare, mi-ar plăcea să găsească mai multe poze, sau utilizatorul să poată selecta ce proprietăți să vadă (ex doar gradul de similitudine, doar histogramele).

Bibliografie

- https://en.wikipedia.org/wiki/Content-based_image_retrieval
- <https://stackoverflow.com/questions/7455646/opencv-how-do-i-split-an-image> - pentru împărțirea imaginii în grid $M \times M$
- <https://users.utcluj.ro/~vcristian/PI.html>
- https://en.wikipedia.org/wiki/Pearson_correlation_coefficient
- https://users.utcluj.ro/~rdanescu/pi_c09.pdf – pentru filtrul Gaussian 5×5